Course Summary & What's Next

Week 13: CS 203 - Software Tools and Techniques for AI

Prof. Nipun Batra

IIT Gandhinagar

The Journey

In January, you knew how to write a Python script.

Now you can:

- **Collect** data from APIs, web pages, and sensors
- **Validate** it with schemas before it poisons your model
- **Label** it efficiently (and know when to use LLMs to help)
- **Augment** it when you don't have enough
- **Evaluate** models properly (not just accuracy!)
- **Tune** hyperparameters (and not waste time doing it)
- **Track** experiments so you never lose a good run
- **Dockerize** your work so anyone can reproduce it
- **Profile** and **optimize** models for real-world constraints

The Full Stack, One Slide

```
Raw Data
   │
   ├── Collect (APIs, scraping, sensors)     ← Week 1-2
   ├── Validate (schemas, types, ranges)     ← Week 2
   ├── Label (manual, active, weak, LLMs)    ← Week 3-4
   ├── Augment (transforms, synthetic)       ← Week 5
   │
   ├── Call LLM APIs (Gemini, prompts)       ← Week 6
   ├── Train & Evaluate (CV, bias-variance)  ← Week 7
   ├── Tune & Track (Optuna, TrackIO)        ← Week 8
   │
   ├── Reproduce (Docker, seeds)             ← Week 9
   ├── Monitor (drift detection)             ← Week 10
   ├── Profile & Optimize (INT8, ONNX)       ← Week 11
   ├── Build Agents (tools, loops)           ← Week 12
   │
   └── Production ML System
```

Week 1-2: Data Collection & Validation

Data Collection: What We Learned

APIs — structured data from services (JSON, REST)

```
response = requests.get("https://api.example.com/data")
data = response.json()
```

Web Scraping — extracting data from HTML pages

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
titles = soup.find_all("h2")
```

Key insight: The data you collect determines your model's ceiling.

No algorithm can fix bad data.

Data Validation: What We Learned

Pydantic — validate data with Python type hints

```
class SensorReading(BaseModel):
    temperature: float = Field(ge=-50, le=60)
    humidity: float = Field(ge=0, le=100)
    timestamp: datetime
```

Key insight: Validate at the boundary. Catch bad data before it enters your pipeline, not after your model produces garbage predictions.

Week 3-5: Data Labeling & Augmentation

Labeling: The Bottleneck

Strategy	When to Use
Manual labeling	Small datasets, high-stakes domains
Active learning	Large unlabeled pool, expensive labels
Weak supervision	Heuristics + noisy labels at scale
LLM-assisted	Text tasks, when budget allows API calls

Key insight: Labeling is usually the most expensive part of ML. The right strategy can save weeks of work.

Augmentation: More Data from Existing Data

Domain	Techniques
Images	Flip, rotate, crop, color jitter
Text	Synonym replacement, back-translation, paraphrase
Tabular	SMOTE, noise injection

Key insight: Augmentation is free data. But it must be *semantically valid* — flipping a 6 to make a 9 doesn't help a digit classifier.

Week 6-8: LLMs, Evaluation & Tuning

LLM APIs: What We Learned

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents=prompt
)
```

Key insight: LLMs are tools, not magic. Prompt engineering, structured outputs, and knowing when NOT to use an LLM are the real skills.

Evaluation: Beyond Accuracy

Concept	Why It Matters
Cross-validation	One train/test split is a coin flip
Stratified K-Fold	Preserves class balance in each fold
Bias-variance tradeoff	Underfitting vs overfitting, visualized
Leakage	The #1 cause of "too good to be true" results

Key insight: If you only do one thing, do **5-fold cross-validation** instead of a single train/test split. It's the minimum viable evaluation.

Tuning: Don't Waste Time

Method	Tries	Finds Best?	When to Use
Grid Search	All combos	Eventually	< 3 hyperparameters
Random Search	Random subset	Often faster	3+ hyperparameters
Bayesian (Optuna)	Smart picks	Yes	Expensive evaluations

Key insight: Random search beats grid search in almost all cases.

Don't tune what doesn't matter.

Week 9-10: Reproducibility & Monitoring

Reproducibility: Three Levels

Level	Problem	Solution
The Math	Different results every run	`random_state=42`
The Memory	Lost the best run	TrackIO
The Machine	Works on my laptop only	Docker

```
# Seeds
X_train, X_test = train_test_split(X, y, random_state=42)

# Tracking
trackio.init(project="my-project", config={...})
trackio.log({"accuracy": 0.95})

# Docker
# docker build -t my-app . && docker run -p 7860:7860 my-app
```

Data Drift: Is Your Model Still Good?

Models degrade over time as the real world changes.

Test	What it detects
KS test	Distribution shift in continuous features
PSI	Population stability between train and production
Chi-squared	Categorical feature changes

Key insight: A model that was 95% accurate at deployment can silently drop to 70% if the input distribution shifts. Monitor continuously.

Week 11: Profiling & Quantization

Making Models Fast and Small

Profiling: Find the bottleneck before optimizing.

```
time.time() → cProfile → line_profiler → memory_profiler
```

Quantization: Make models smaller with minimal accuracy loss.

```
FP32 (4 bytes) → INT8 (1 byte) = 4x smaller
```

ONNX: Train in Python, run anywhere (mobile, browser, edge).

Key insight: The #1 speedup is loading the model once at startup instead of per-request. Profile before you optimize.

Week 12: Building AI Agents

Agents: LLMs That Take Actions

An **agent** = LLM + tools + a loop.

Piece	What it does	Analogy
LLM	Thinks, reasons, decides	The brain
Tools	Functions the LLM can call	The hands
Loop	Keeps going until done	The work ethic

You built a complete agent with **Gemma 4** on a free Colab T4: four tools, multi-step reasoning, ~100 lines of Python.

Key insight: Claude Code, Cursor, Devin, Perplexity — they're all the same pattern. LLM + tools + loop. The tools are bigger, but the architecture is identical to what you built.

Connecting the Dots

The Tools Map

```
PLAN            BUILD           SHIP            EXTEND
────            ─────           ────            ──────
Validation      sklearn         Docker          cProfile
Schemas         Pipelines       TrackIO         Quantization
Labeling        CV/Tuning       Seeds           ONNX
Augmentation    Optuna          Drift detect    Agents
                LLM APIs                        Tool calling
```

You don't need all of these for every project. But you need to know they exist so you can pick the right tool for the job.

The Minimum Viable ML Project

For your course project, this is the minimum:

```
my-project/
├── data/                    # raw + processed data
├── notebooks/               # exploration
├── src/
│   ├── train.py            # training script with random_state
│   └── app.py              # Gradio/Streamlit app
├── requirements.txt         # pinned versions
├── Dockerfile               # optional but impressive
└── README.md                # how to run it
```

With:

- Seeds set everywhere
- At least 5-fold CV
- TrackIO logging

What This Course Didn't Cover (But You Should Know Exists)

Topic	Why It Matters	Where to Learn
Deep Learning	CNNs, transformers, fine-tuning	CS 337, fast.ai
MLOps	CI/CD, model registries, pipelines	Made With ML
Cloud Deployment	AWS/GCP/Azure, Kubernetes	Cloud provider docs
Data Engineering	ETL pipelines, data warehouses	dbt, Airflow
ML System Design	Scaling, A/B testing, feedback loops	Chip Huyen's book
MCP / A2A	Universal tool standards for agents	Anthropic docs, Google docs

Advice for Your Future Projects

1. **Start with the data, not the model.** 80% of ML is data work.
2. **Simple models first.** Logistic regression before transformers. You'd be surprised how often simple wins.
3. **Automate the boring stuff.** Seeds, tracking, Docker, requirements.txt — set these up on day 1, not day last.
4. **Make it runnable.** If someone can't clone your repo and run your code in 5 minutes, it doesn't exist.
5. **Read error messages.** The answer is almost always in the traceback. Read it before asking ChatGPT.

Thank You

This was CS 203: **Software Tools and Techniques for AI**.

You started with scripts. You end with deployable, reproducible, optimized ML systems — and agents that can take actions.

Go build something.

```
Seeds  →  TrackIO  →  Docker  →  Agents  →  Production
  ↓          ↓          ↓          ↓           ↓
Control   Remember   Reproduce   Act        Ship
```

Questions?

From scripts to systems. Go build.